Table 4.1 Each PEP deployment option balances control, consistency, and complexity in different ways. Choosing the appropriate pattern depends on your architecture, the required granularity of access control, and the necessity for centralized policy decisions.

Deployment model	Pros	Cons	Best used when?
Embedded in application	Fine-grained control Access to full application context Simple for single-service apps	Repetition across apps Harder to update uniformly	You need contextual logic tied closely to app behavior. For example, use cases requiring high ,security such as financial payment applications might use this model.
Embedded in API gateway	Centralized enforcement across services Easy to update Built-in logging/auditing	Limited to API request-level policies Less context about application internals	You want consistent checks with minimal app changes. For example, if your product is available as an API, putting access control logic in the gateway provides consistent and flexible application of policy.
Deployed in sidecar container	Works across microservices Enforces policies between services Invisible to app code	Operational overhead Caching/staleness risks	You're running a service mesh or need zero-trust enforcement. For example, if your application is made of many microservices, sidecars allow service-specific policies for fine-grained access control.

When the PEP is embedded in application code, developers gain access to the full context of the request, which facilitates fine-grained, dynamic access decisions. This is useful for apps like ACMEProject, where access to specific UI elements or resource attributes may depend on user roles, project assignments, or workflow status. However, this approach can result in duplicated logic across services and makes centralizing policy updates more challenging.

By contrast, placing the PEP in an API gateway provides a centralized enforcement point for all incoming requests. This approach simplifies